System Architecture Specification

CNC Tool Manager Application

This specification document outlines the technical architecture, database schema, user interface layout, integration endpoints, and deployment workflow for the CNC Tool Manager web application.

---

1. Tech Stack Overview

The application architecture relies on a modern TypeScript stack, ensuring high type safety, performance, and maintainability across both client and server domains.

- **Framework:** Next.js (App Router, Server Actions, API Routes)
- **UI Library:** React with Tailwind CSS for component-level styling
- **Database ORM:** Drizzle ORM
- **Database Provider:** Supabase (PostgreSQL hosting)
- **State & Interactivity:** React HTML5 Drag and Drop (or `@hello-pangea/dnd`) for interactive tool slotting

---

2. Database Schema (Drizzle ORM)

The relational schema centers on tool definitions, magazine physical mapping, and purchasing audit logs.

Schema Definitions

```
import { pgTable, uuid, text, numeric, integer, timestamp } from 'drizzle-orm/pg-core';

export const tools = pgTable('tools', {

  id: uuid('id').defaultRandom().primaryKey(),

  tnum: text('tnum').notNull(), // User-facing tool identifier

  description: text('description'),

  shankIn: numeric('shank_in'),

  shankM: numeric('shank_m'),

  cutDia: numeric('cut_dia'),
```

```
  loc: numeric('loc'), // Length of cut
  type: text('type'),
  part: text('part'),
  series: text('series'),
  coating: text('coating'),
  hsm: text('hsm'),
  radius: numeric('radius'),
});
export const magazines = pgTable('magazines', {
  id: uuid('id').defaultRandom().primaryKey(),
  machineName: text('machine_name').notNull(),
  totalPockets: integer('total_pockets').default(21).notNull(),
});
export const magazinePockets = pgTable('magazine_pockets', {
  id: uuid('id').defaultRandom().primaryKey(),
  magazineId: uuid('magazine_id').references(() => magazines.id, { onDelete: 'cascade' }),
  pocketNumber: integer('pocket_number').notNull(), // 1 to 21
  toolId: uuid('tool_id').references(() => tools.id, { onDelete: 'set null' }),
});
export const purchases = pgTable('purchases', {
  id: uuid('id').defaultRandom().primaryKey(),
  toolId: uuid('tool_id').references(() => tools.id, { onDelete: 'cascade' }),
  date: timestamp('date').defaultNow().notNull(),
  vendor: text('vendor'),
```

```
  price: numeric('price'),
  poNumber: text('po_number'),
  quantity: integer('quantity').default(1).notNull(),
});
```

Table Summary

Table	Primary Purpose	Key Foreign Keys
`tools`	Master catalog for physical and geometric tool attributes	None
`magazines`	Represents CNC physical tool changers	None
`magazine_pockets`	Slot-by-slot index of current physical tool loadouts	`magazine_id, tool_id`
`purchases`	Inventory procurement log (Cutterlog)	`tool_id`

---

3. UI & Views

The front-end user experience is divided into two core views designed for shop-floor management and tool configuration.

3.1 Form View

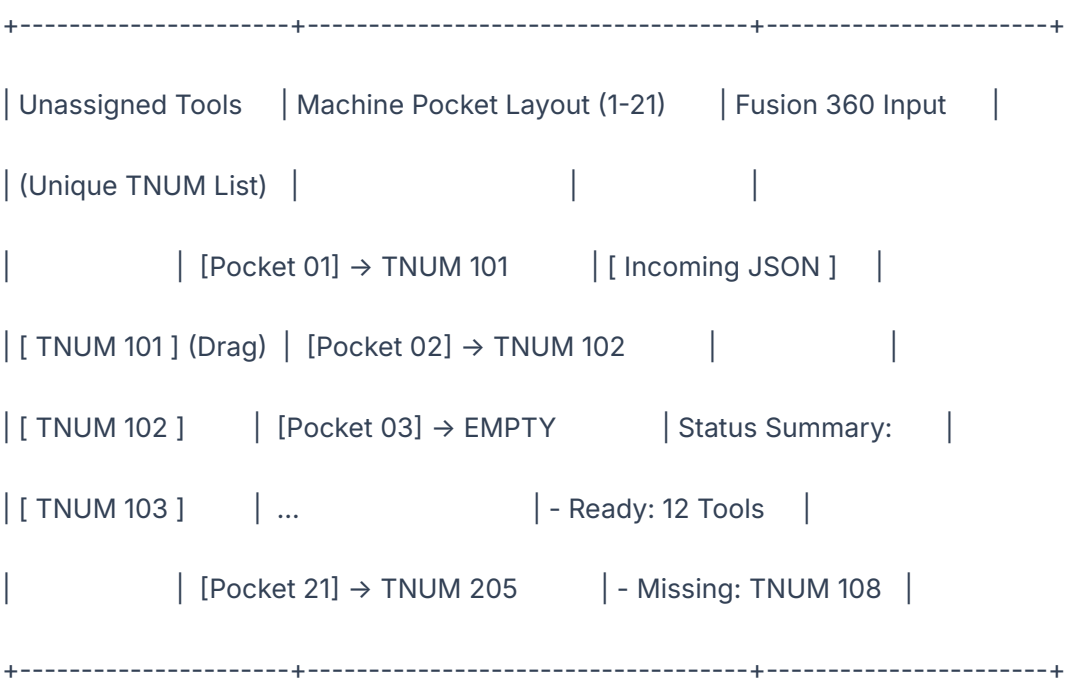
The **Form View** acts as the data entry portal for adding and editing tool specifications into the `tools` table and recording restock events into the `purchases` table.

- Input fields mapped directly to geometric values (`cut_dia`, `loc`, `shank_in`, `shank_m`, `radius`).
- Dropdown selectors for categorical attributes (`type`, `coating`, `hsm`, `series`).
- Purchase entry section allowing immediate creation of a linked `purchases` log entry upon tool receiving.

3.2 Magazine View

The **Magazine View** provides an interactive visual interface representing the machine tool changer.+----------+

| MAGAZINE VIEW |



- **Pocket Grid (Center):** Displays pockets 1 through 21 for the active machine magazine. Supports dropping tools into pockets and swapping tool slots via drag-and-drop.
- **Available Tools Panel (Left):** Lists all defined tools from the database, grouped to display **one row per unique `tnum`**. Items can be dragged directly into a target pocket.
- **Fusion 360 Verification Panel (Right):** A specialized "square" or status panel displaying real-time or last-received tool list data sent from Fusion 360.
    - **Matched Tools:** Highlighted green if the required `tnum` is currently mounted in a `magazine_pocket`.
    - **Missing Tools:** Highlighted red/amber if required by the active CAM operation but missing from the magazine pockets.

---

4. Autodesk Fusion Integration

The system integrates with Autodesk Fusion 360 post-processors and scripts to synchronize tool requirements with physical setup states.

4.1 Inbound Tool Sync Endpoint

An API route receives the JSON payload generated by the Fusion 360 script when a user posts an operation or sends a tool table setup.

- **Endpoint:** `POST /api/fusion-tools`
- **Payload Format:**

```
{
  "machineName": "VF-2",
```

```
  "requiredTools": [
    { "tnum": "101", "description": "0.500 Flat Endmill" },
    { "tnum": "108", "description": "0.250 Chamfer Mill" }
  ]
}
```

- **Processing Pipeline:**
  a. Validate request body against schema.
  b. Broadcast/push updated JSON state to the client (via SWR polling or WebSockets).
  c. Update the **Fusion 360 Verification Panel** state in the Magazine View.
  d. Execute diff against active `magazine_pockets` records to highlight tool discrepancies.

4.2 Bidirectional Extension Placeholder

A bidirectional synchronization stub is implemented in `POST /api/fusion-tools/export`. This allows future capability for the web application to transmit assigned pocket allocations back into Fusion 360 Tool Library formats (`.json` / `.hsmlib`).

---

5. Deployment & Environment Setup

The application is configured for local execution on a macOS host within the shop network, connecting to cloud-hosted database infrastructure.

- **Host Environment:** Node.js (v18+ LTS) on Mac OS
- **Process Management:** `pm2` or native macOS `launchd` service for background execution and auto-restart on boot.
- **Database Host:** Supabase Cloud PostgreSQL

Environment Variables (`.env.local`)

```
# Database Connections
DATABASE_URL="postgresql://postgres:[PASSWORD]@db.[SUPABASE_REF].supabase.co:5432/postgres"

# Application Settings
NEXT_PUBLIC_APP_URL="http://localhost:3000"
FUSION_API_SECRET="internal-api-key-for-fusion-script"
```